SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I SHAILAJA S(192511026) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

The Water Jug Problem is a classic example used in Artificial Intelligence (AI) to demonstrate analytical problem solving and state space search techniques.

- **State Space Representation**

  Each state is represented as **(x, y)**

  - x = amount of water in 4-gallon jug
  - y = amount of water in 3-gallon jug

- **Initial State**

  - Both jugs are empty → **(0, 0)**

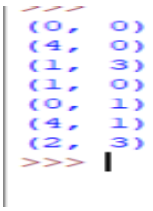
- **Goal State**

  - Required result → **(2, 0)**

CODE:

```
from collections import deque

def jug():
    q = deque([((0,0), [])])
    visited = set()

    while q:
        (x,y), path = q.popleft()
        if (x,y) in visited:
            continue

        visited.add((x,y))
        path = path + [(x,y)]

        if x == 2:
            return path

        for nx, ny in [
            (4,y), (x,3), (0,y), (x,0),
            (x-min(x,3-y), y+min(x,3-y)),
            (x+min(y,4-x), y-min(y,4-x))
        ]:
            q.append(((nx,ny), path))

for step in jug():
    print(step)
```

OUTPUT:



2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

A Mars Rover is an intelligent agent designed to explore the Martian surface, collect samples, analyse them, and send data back to Earth. In AI terms, it interacts with its environment through percept's (inputs) and actions (outputs) to achieve mission goals.

I. Percepts are the inputs the rover receives from its sensors:

- Visual data from cameras (terrain, rocks, obstacles)
- Temperature and atmospheric conditions
- Soil composition and chemical analysis readings
- Distance measurements from sensors (obstacle detection)
- Internal system status (battery level, faults)

II. The Mars Rover operates in a challenging and uncertain environment:

- **Partially observable** – cannot sense everything at all times
- **Deterministic/Stochastic** – some actions have uncertain outcomes (e.g., wheel slipping)
- **Sequential** – current actions affect future states
- **Dynamic** – environment can change (dust storms, terrain shifts)
- **Continuous** – movement and time are continuous
- **Hostile environment** – extreme temperatures, radiation, rough terrain

III. The rover can perform various actions:

- Move forward, backward, or turn
- Capture images and videos
- Drill rocks and collect soil samples
- Analyze samples using onboard instruments
- Transmit data to Earth
- Adjust solar panels or conserve energy

*IV. **Performance is measured based on how well the rover achieves its mission goals:***

- Accuracy of collected scientific data
- Number and quality of samples analyzed
- Successful navigation without damage
- Efficient energy usage (battery management)
- Duration of operation (mission lifespan)
- Reliability in harsh conditions

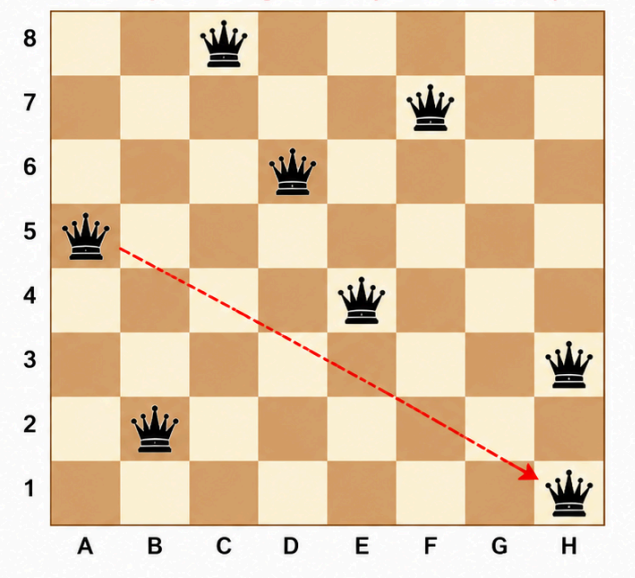
*V. **Hybrid Agent Architecture:***

- **Model-Based** → Maintains internal map of surroundings
- **Goal-Based** → Focuses on mission objectives (exploration, sample collection)
- **Utility-Based** → Chooses best action considering energy, safety, and priority

Why Hybrid?

- Mars environment is **complex and unpredictable**
- Rover must **plan, adapt, and optimize decisions**
- Combines reasoning, learning, and decision-making capabilities

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.



- The 8-Queens problem involves placing eight queens on an 8×8 chessboard so that no two queens attack each other. The search space includes all possible queen arrangements, but only those satisfying the constraints (no same row, column, or diagonal) are valid. Each state represents positions of queens placed so far, starting

from an empty board (initial state) and reaching a goal state when all 8 queens are placed without conflict. The given configuration is not a solution because some queens lie on the same diagonal.

- To solve this problem, search strategies like Backtracking, DFS, and Heuristic methods are used. Backtracking places queens step-by-step and removes them if conflicts occur. The problem formulation includes state space, operators (placing queens), goal test (no attacks), and path cost (not important). This problem demonstrates how AI solves constraint satisfaction problems using systematic search.

CODE:

```
def solve(n, col=0, board=[]):
    if col == n:
        print(board)
        return True

    for row in range(n):
        if all(board[i] != row and
               abs(board[i] - row) != col - i for i in range(col)):
            solve(n, col + 1, board + [row])

solve(8)
```

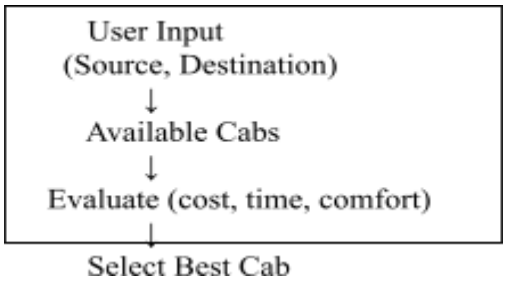
OUTPUT:

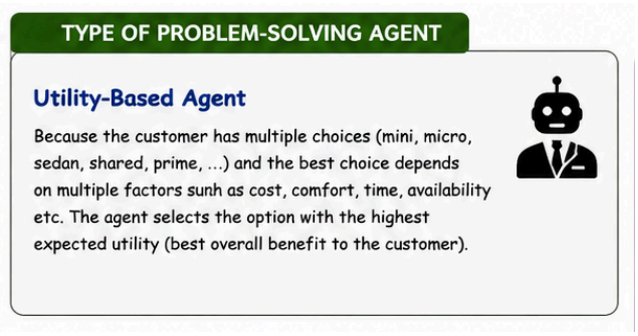
```
[0, 4, 7, 5, 2, 6, 1, 3]
```

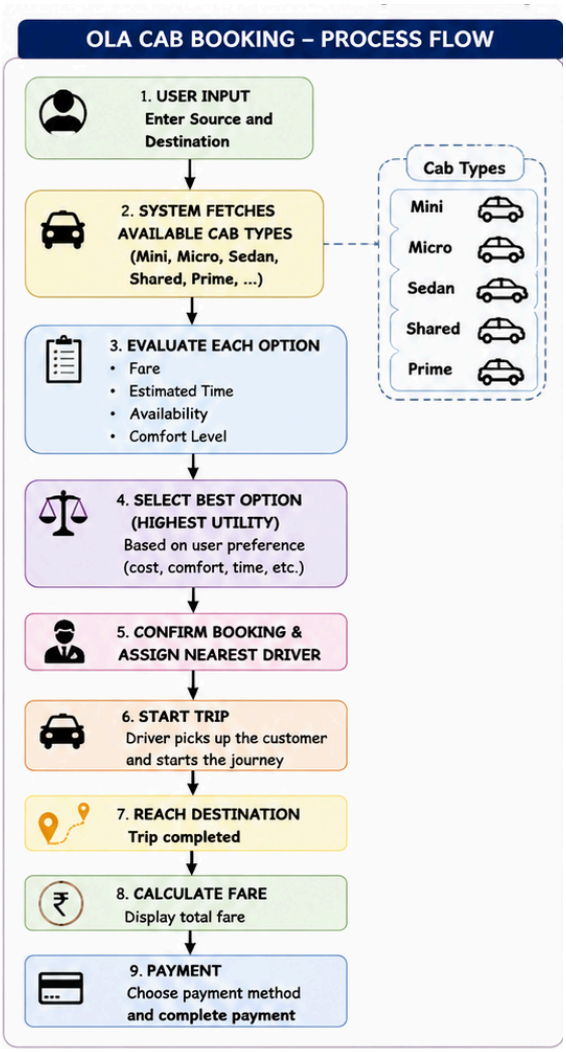
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

The suitable agent is a Utility-Based Agent.

- The user has **multiple choices** (mini, micro, sedan, shared, prime)
- Each option differs in **cost, comfort, time, availability**
- The agent selects the **best option based on user preference (utility)**





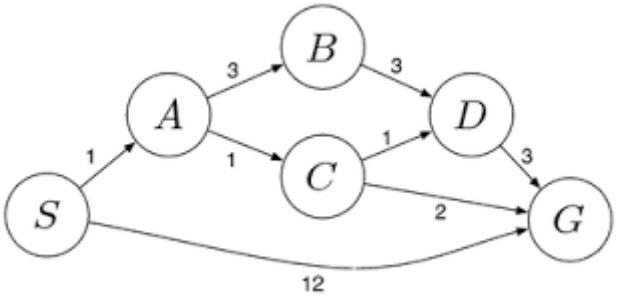


PSEUDOCODE:

```
START

INPUT source location
INPUT destination location

DEFINE cab types = {mini, micro, sedan,
shared, prime}

best utility ← -∞
best cab ← NULL

FOR each cab IN cab types DO
   fare ← calculate fare (cab, source location,
destination location)
   time ← estimate time (cab, source location,
destination location)
   availability ← check availability(cab)
   comfort ← get_comfort_level(cab)

   utility ← evaluate (fare, time, availability,
comfort)

   IF utility > best utility THEN
      best utility ← utility
      best cab ← cab
   ENDIF
ENDFOR

CONFIRM booking of best_cab

driver ← assign_nearest_driver(best_cab,
source_location)
```

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



A weighted graph representing a delivery network with the following edge costs:

- S → A = 1
- S → G = 12
- A → B = 3
- A → C = 1
- B → D = 3
- C → D = 1
- C → G = 2
- D → G = 3

Uniform Cost Search (UCS) Steps:

- Start at node **S** with cost = 0
  Frontier: A (1), G (12)
- Expand **A (1)**
  Frontier: C (2), B (4), G (12)
- Expand **C (2)**
  Frontier: D (3), G (4), B (4), G (12)
- Expand **D (3)**
  Frontier: G (4), B (4), G (6), G (12)
- Expand **G (4)** → Goal reached

Optimal Path:
S → A → C → G

Total Cost:
1 + 1 + 2 = 4

Step	Node Expanded	Path	Cumulative Cost (g)	Frontier (Nodes with Cost)
0	Start at S	S	0	A(1), G(12)
1	Expand A (1)	S → A	1	C(2), B(4), G(12)
2	Expand C (2)	S → A → C	2	D(3), G(4), B(4), G(12)
3	Expand D (3)	S → A → C → D	3	G(4), B(4), G(6), G(12)
4	Expand G (4)	S → A → C → G	4	Goal Reached

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem